Institute of Technology of Cambodia
Faculty of Electrical Engineering
Génie Informatique et Communication

GICI4OS · OPERATING SYSTEMS · LAB 1

Exploring Operating System Basics

Lab 1 guide: Linux in brief, then find out which system you are on, practise the basic commands, and see programs and processes at work.



Heng Rathpisey · Department GIC · Academic year 2025–2026

Learning objectives

1. Identify basic operating system and kernel information.
2. Use essential Linux file and directory commands.
3. Install, remove, and purge software using the APT package manager.
4. Understand the difference between a program and a running process.
5. Observe multitasking in a running operating system.
6. Detect whether an operating system is running on a virtualized environment.

THIS LAB

Lab 1

First, Linux in brief. Then Part A in class, on the course server, and Part B as homework, on your own Ubuntu.

DISCUSSION

You log in to a server you have never seen. What do you check first?

Which system it is, what is in your folder, and what is running. This lab shows you how to find out.

What is Linux, and why learn it?

Where it comes from, how a distribution is built, and how you talk to it.

Linux is everywhere

- Servers and cloud platforms
- Android phones and tablets, which use the Linux kernel
- Routers, smart TVs and cars
- Supercomputers
- Developers' laptops and desktops

As an engineer you will meet Linux again and again. That is why this course works on it.



Where Linux comes from

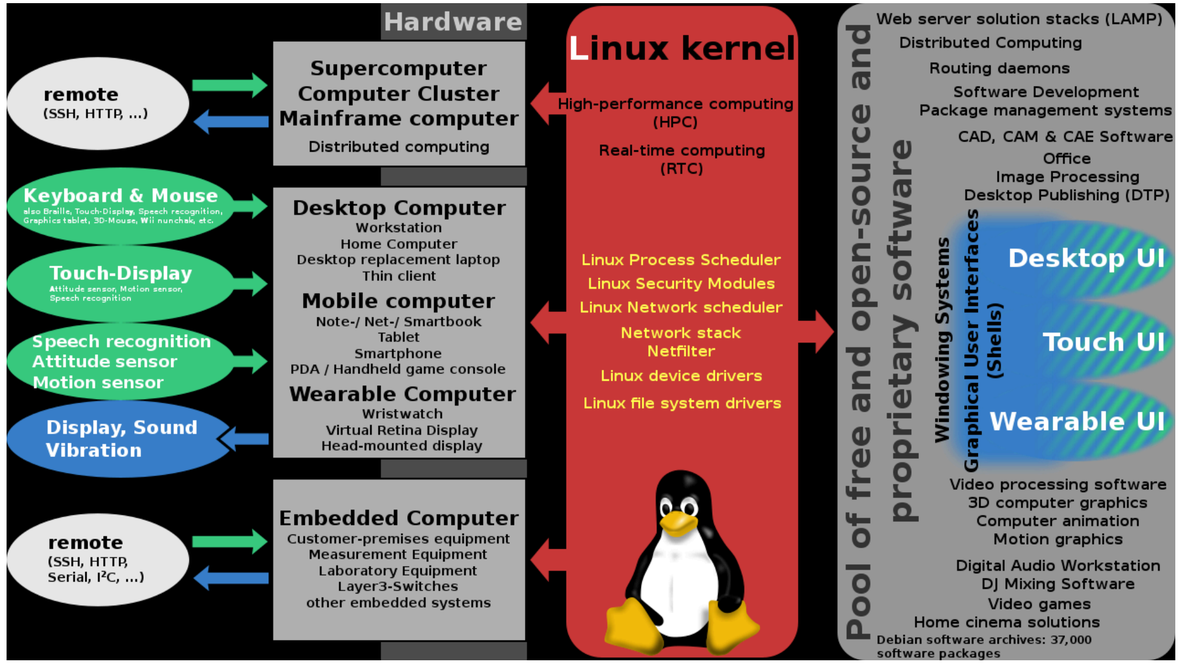
- Linus Torvalds developed the first version, 0.01, in 1991.
- Linux was first distributed as source code only.
- Later it came as a pair of downloadable floppy-disk images.
- One image was bootable and held the Linux kernel.
- The other held a set of GNU utilities and tools for setting up a file system.



Linus Torvalds, 1969 to present

The Linux kernel

- The kernel is the core of the operating system.
- It manages the processor, the memory and the devices.
- Programs ask it for services through system calls.
- Linux is the kernel. A distribution adds the programs around it.

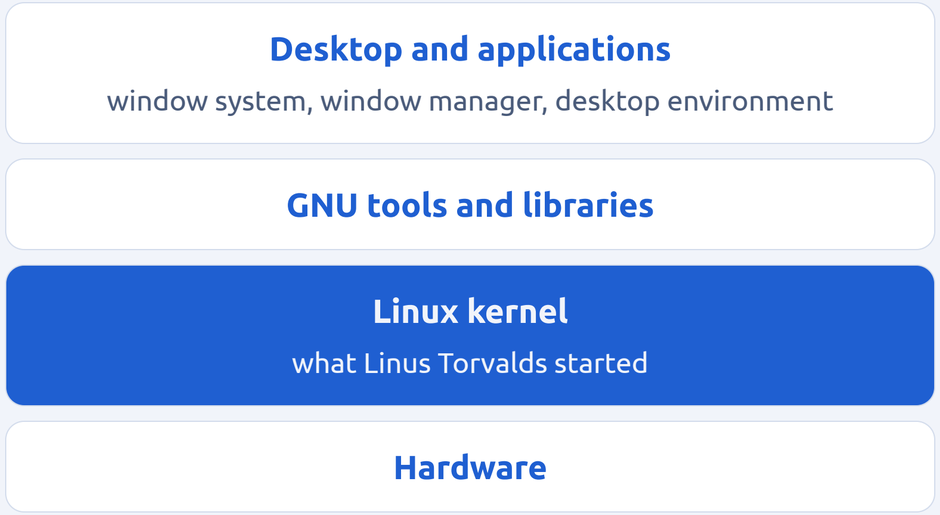


A Linux distribution, or distro

An operating system made from a software collection, based on the Linux kernel and, often, a package management system.

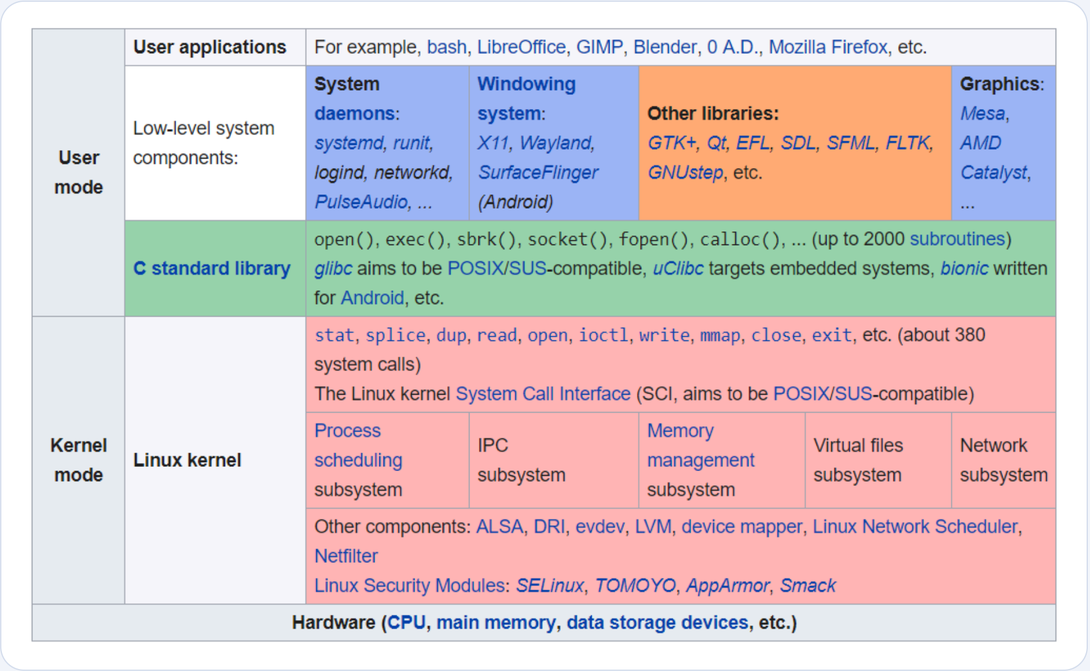
A typical distribution comprises

- A Linux kernel
- GNU tools
- Others: libraries, additional software, documentation, a window system, a window manager and a desktop environment



About 500 to 600 distributions are in active development (the figure given in the earlier Lab 1 guide).

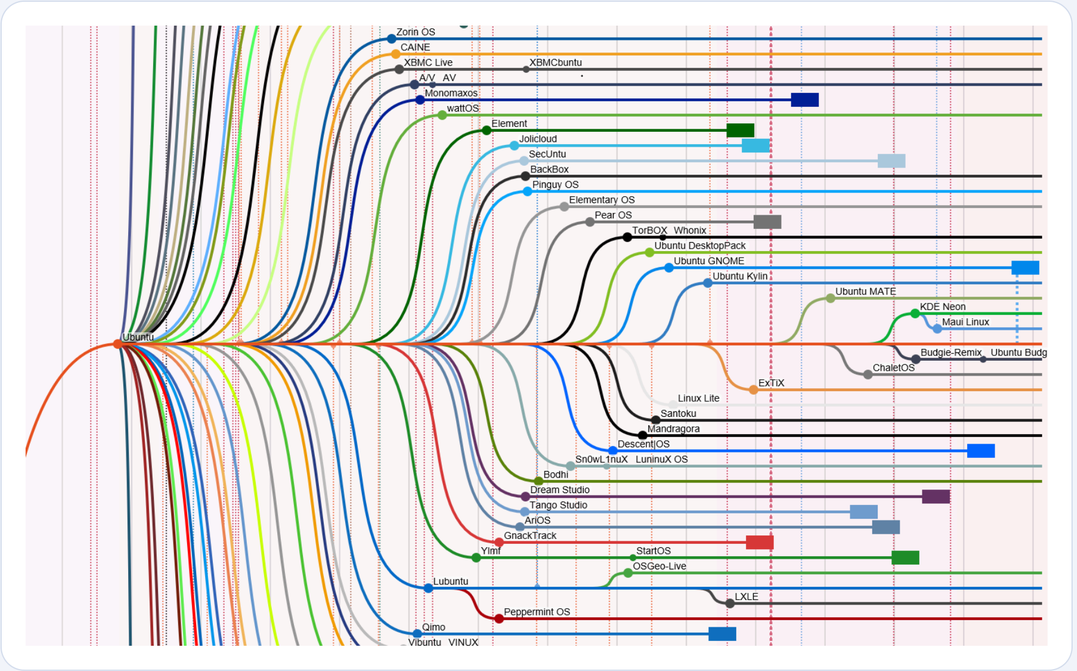
Layers of a Linux distribution



User mode	**User applications**	For example, bash, LibreOffice, GIMP, Blender, 0 A.D., Mozilla Firefox, etc.				
	Low-level system components:	**System daemons**: *systemd*, *runit*, *logind*, *networkd*, *PulseAudio*, ...	**Windowing system**: *X11*, *Wayland*, *SurfaceFlinger* (Android)	**Other libraries:** *GTK+*, *Qt*, *EFL*, *SDL*, *SFML*, *FLTK*, *GNUstep*, etc.	**Graphics**: *Mesa*, *AMD Catalyst*, ...	
	C standard library	`open()`, `exec()`, `sbrk()`, `socket()`, `fopen()`, `calloc()`, ... (up to 2000 subroutines) *glibc* aims to be POSIX/SUS-compatible, *uClibc* targets embedded systems, *bionic* written for Android, etc.				
Kernel mode	**Linux kernel**	`stat`, `splice`, `dup`, `read`, `open`, `ioctl`, `write`, `mmap`, `close`, `exit`, etc. (about 380 system calls) The Linux kernel System Call Interface (SCI, aims to be POSIX/SUS-compatible)				
		Process scheduling subsystem	IPC subsystem	Memory management subsystem	Virtual files subsystem	Network subsystem
		Other components: ALSA, DRI, evdev, LVM, device mapper, Linux Network Scheduler, Netfilter Linux Security Modules: *SELinux*, *TOMOYO*, *AppArmor*, *Smack*				
Hardware (CPU, main memory, data storage devices, etc.)						

Programs in user mode sit on the kernel, and the kernel sits on the hardware.

A timeline of distributions



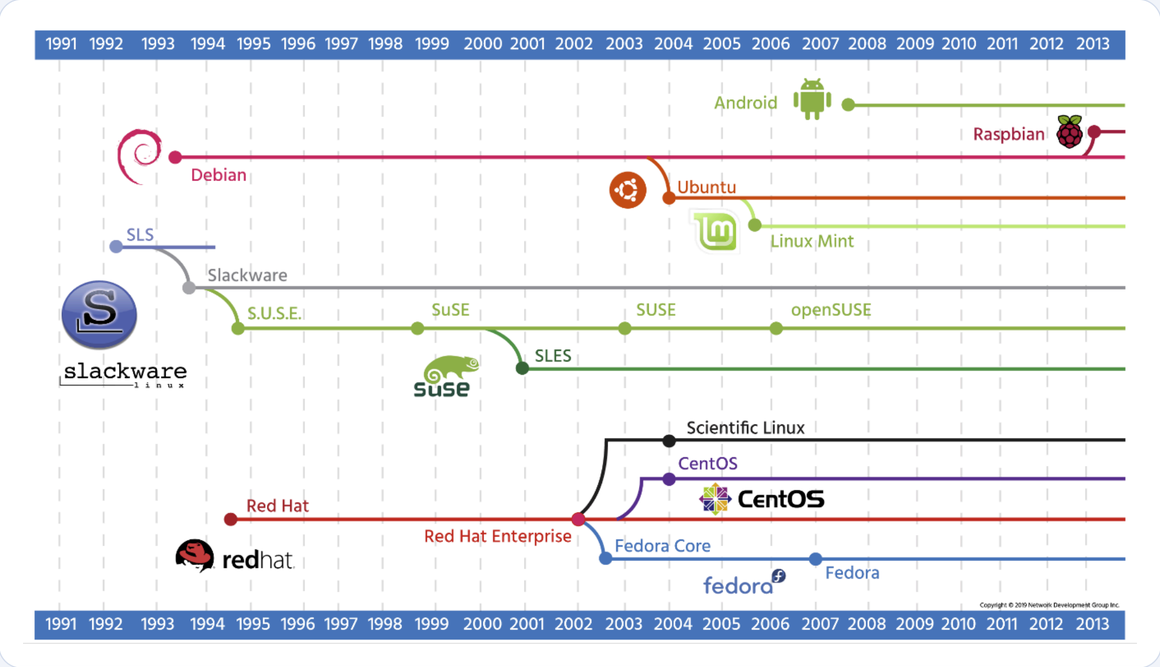
Each line is one distribution. Most of them branch from an older one.

Linux families

Family	Examples	Package files	Tools
Debian	Debian, Ubuntu, Linux Mint, Raspberry Pi OS	`.deb`	`dpkg, apt`
Red Hat	Red Hat Enterprise Linux, Fedora, CentOS	`.rpm`	`rpm, yum, dnf`
SUSE	SUSE Linux Enterprise, openSUSE	`.rpm`	`rpm, zypper`
Slackware	Slackware	`.tgz`	`pkgtools`

This lab uses Ubuntu, so you use the tools of the Debian family: dpkg and apt.

The families over time



Debian, Slackware, SUSE and Red Hat each started a family of distributions.

Package management

01

Packages

A distribution is normally split into packages. Each package holds one application or service.

02

What a package holds

The software, and metadata: its full name, a description of its purpose, a version number, a checksum, and the list of dependencies it needs to run.

03

The package manager

A collection of tools that automates installing, upgrading, configuring and removing programs, in a consistent way.

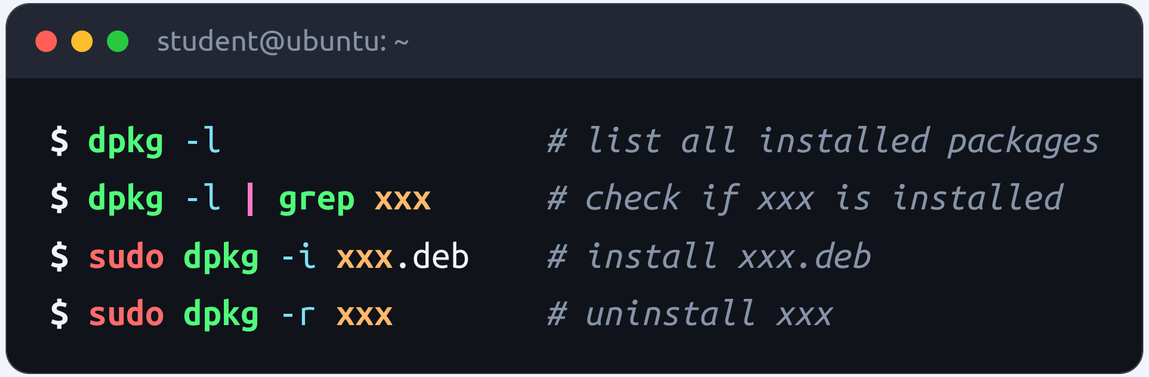
A dependency is another package that this one needs.

DPKG

DPKG is the package manager of Debian-based systems.

It can install, remove and build packages. Unlike other package managers, it cannot download and install packages or their dependencies by itself.

Details: man dpkg

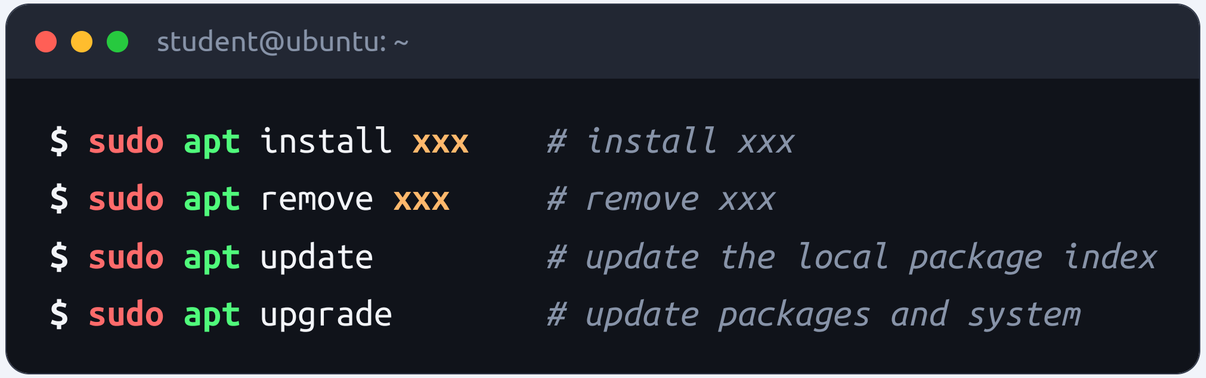


APT

The apt command works with Ubuntu's Advanced Packaging Tool.

It installs new packages, upgrades the ones you have, updates the package list, and can even upgrade the whole system.

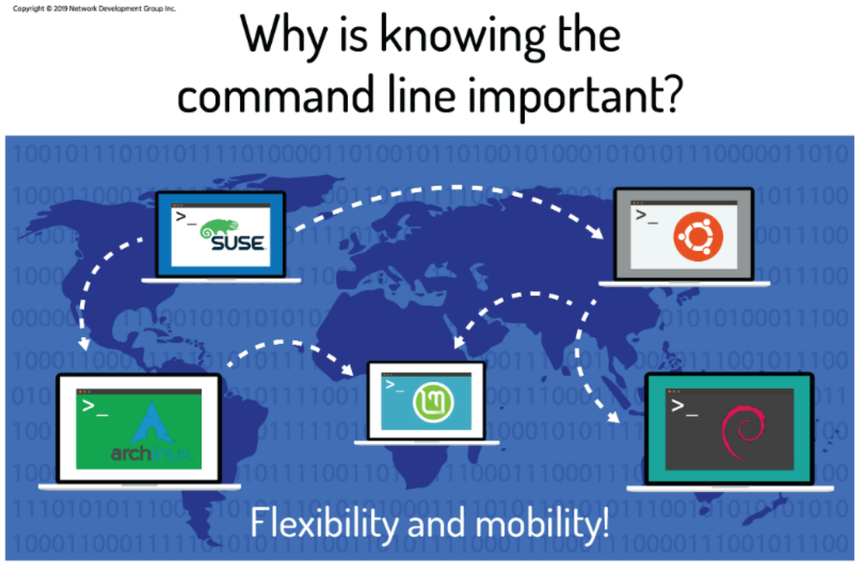
Details: apt help. Lab 1 uses apt-get, the older command with the same actions.



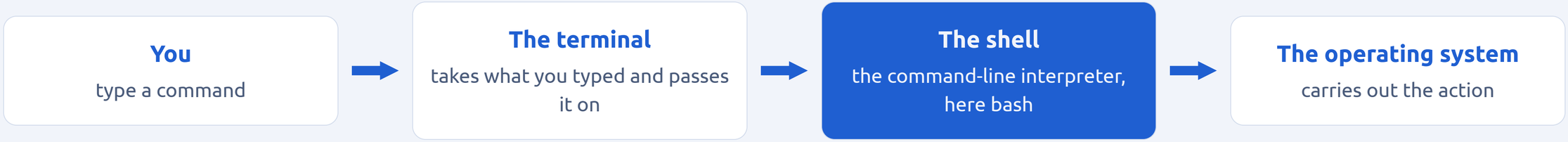
Why is knowing the command line important?

Flexibility and mobility.

- When you understand the foundation of Linux, you can work on any Linux distribution.
- That may be one company with a mixed environment.
- Or a new company that uses a different distribution.
- Many servers have no graphical desktop at all.



What is a shell?



The terminal

Once you have entered a command, the terminal accepts what you typed and passes it to a shell.

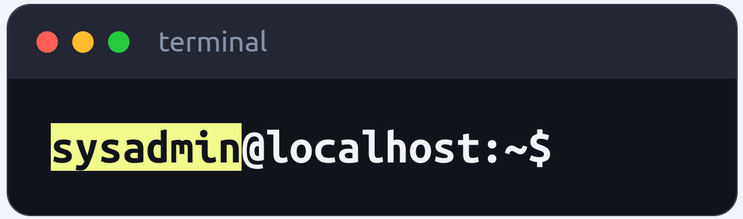
The shell

The shell is the command-line interpreter. It translates the commands you enter into actions to be performed by the operating system.

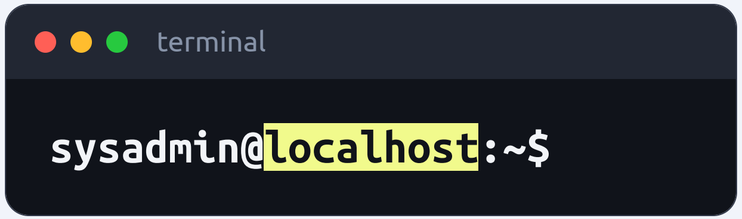
The prompt

When a terminal starts, the shell shows a prompt. Its structure varies between distributions, but it typically tells you about the user and the system.

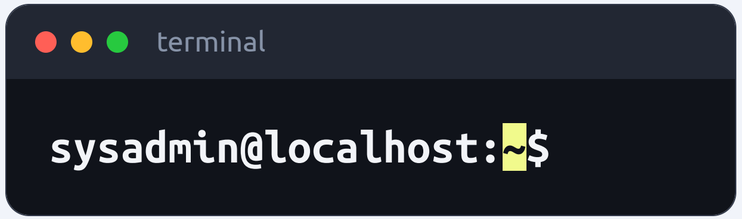
User name



System name

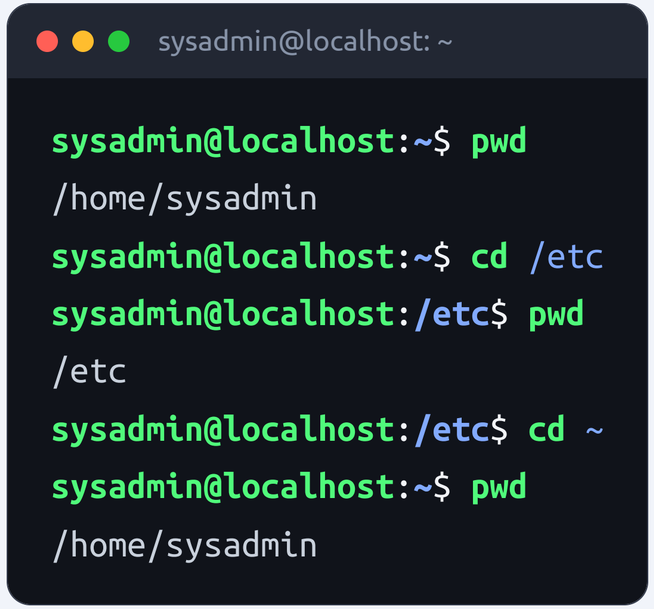


Current directory



The $ at the end means you are a normal user, and the shell is waiting for your command.

The ~ symbol is your home directory



The ~ symbol is shorthand for the user's home directory.

Watch the prompt: it shows ~ while you are at home, and the real path when you are somewhere else.

cd ~ brings you home from anywhere.

LAB 1

Exploring operating system basics

Two parts: in class on the server, then homework on your own Ubuntu.

Two parts, one lab

WITH THE INSTRUCTOR

Part A: in class

120 minutes on the course server. Five tasks, a prediction and a live checkpoint. Worth 7 of the 10 points.

DUE BEFORE THE NEXT CLASS

Part B: homework

About 60 minutes on your own Ubuntu and PC. Install and remove software, three screenshots, your report and a push to GitHub. Worth 3 points.

Both parts are graded together as Lab 1.

Where each task runs

Task	What you do	Where
1	Find the kernel and the distribution	Server, in class
2	Practise file and folder commands	Server, in class
3	Install, remove and purge a package	Your Ubuntu, homework
4	Program or process?	Server, in class
5	Several programs at once	Server, in class
6	Is this a virtual machine?	Server, in class

You have no sudo on the server, so only Task 3 happens on your own machine.

Inside the 120 minutes

Minutes	What happens
0 to 10	Setup
10 to 20	Task 1: which system is this?
20 to 40	Task 2: files and folders
40 to 50	Prediction, then class discussion
50 to 65	Tasks 4 and 5: programs, processes and multitasking

Minutes	What happens
65 to 75	Task 6: is this a virtual machine?
75 to 90	Instructor demonstration, then Plus and Challenge
90 to 105	Live checkpoint
105 to 120	Debrief and homework briefing

Five helper commands, and why you use them

Command	What it does	Why
`oslab values lab1`	Shows your own names and number	Your work is different from your neighbour's
`oslab predict lab1`	Saves your guess before you try	Guessing first makes you think. The first answer is kept
`oslab hint lab1 1`	A hint, levels 1 to 3	You get unstuck without waiting, at no cost
`oslab check lab1`	Checks your files	You see at once what is still missing
`oslab checkpoint lab1`	A short test near the end	It shows what you can do alone

How you work

Individual work

Your file names and number are different from your neighbour's. You hand in your own work.

AI tools

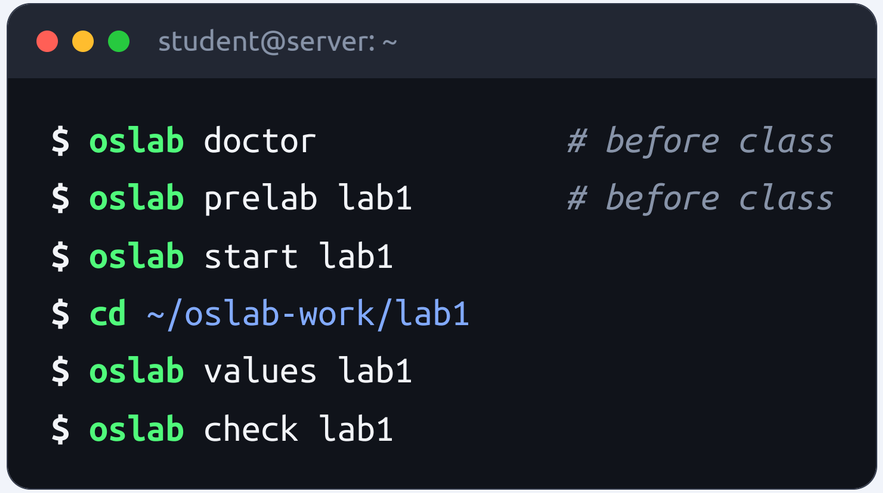
Allowed in the tasks, so test what they tell you. Not allowed for the prediction and the checkpoint.

Hints

Use oslab hint lab1, levels 1 to 3, before you ask. Hints cost no points.

A wrong prediction costs nothing. It is marked for being made and corrected, not for being right.

Start the lab



- oslab start lab1 makes your lab folder.
- cd takes you into it. You work there for all of Part A.
- oslab values lab1 prints your personal values. The next slide explains them.
- oslab check lab1 prints one line for each part of the lab: TRY means not done yet, PASS means done.

At the start every line of oslab check lab1 says TRY, because you have not done anything yet.

Your personal values

1. The command prints values that are only yours

example: your values will be different

```
$ oslab values lab1
Your personal values for lab1 (your-account):
  count    = 3
  file1    = maple
  file2    = river
```

2. The guide writes capitals where your value belongs

The guide says	With the example values, you type
`touch FILE1.txt FILE2.txt`	`touch maple.txt river.txt`
`echo "This is file FILE1"`	`echo "This is file maple"`
Type this line `COUNT` times	Type this line 3 times

FILE1, FILE2 and COUNT are placeholders, shown in orange in the terminal windows. Never type them. Type your own value in their place.

TASK 1

Which system is this?

The kernel and the distribution are not the same thing.

Ask the system about itself

student@server: ~/oslab-work/lab1

```
$ uname -a > task1_os_info.txt
$ lsb_release -a >> task1_os_info.txt
$ cat task1_os_info.txt
```

uname asks the kernel. lsb_release describes the distribution.



For your report: which number is the kernel version, and which is the Ubuntu version?

TASK 2

Files and folders

Ten commands, and two file names that are only yours.

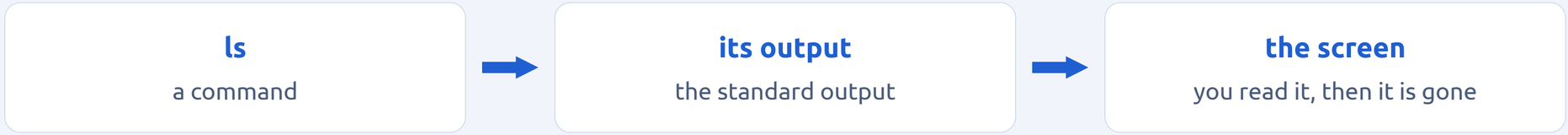
The ten commands

Command	Meaning
`pwd`	Shows the folder you are in
`ls`	Lists the files here
`mkdir`	Makes a folder
`cd`	Goes into a folder
`touch`	Makes an empty file

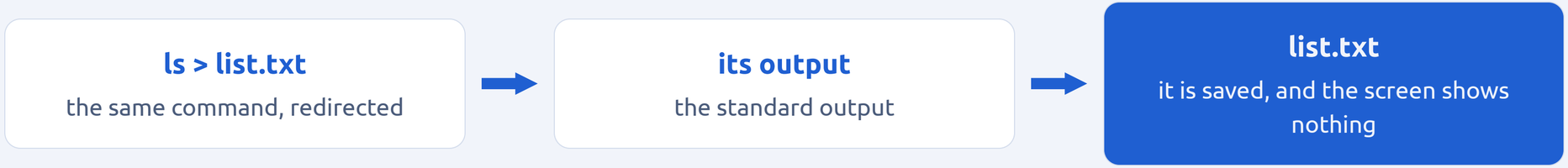
Command	Meaning
`echo`	Prints text
`cat`	Shows what is in a file
`cp`	Copies a file
`mv`	Moves or renames a file
`rm`	Deletes a file

Where does the output of a command go?

Normally: to the screen



With > or >>: into a file



This is called output redirection. You use it in every task to keep a record of your results.

The syntax of > and >>

You type	What it means	If the file is new	If the file exists
`COMMAND > FILE`	Send the output into FILE	FILE is created	Its old content is replaced
`COMMAND >> FILE`	Add the output to the end of FILE	FILE is created	Its old content stays, and the new output comes after it

One arrow replaces

Think of > as: start this file again.

Two arrows add

Think of >> as: write on the next line.

The command does not change

Only where its output goes changes.
COMMAND can be any command.

See the difference

> replaces the file

one arrow

```
$ echo one > a.txt
$ echo two > a.txt
$ cat a.txt
two
```

>> adds to the end

two arrows

```
$ echo one > a.txt
$ echo two >> a.txt
$ cat a.txt
one
two
```

In this lab the first line of a record file uses > and every later line uses >>.

Two dots mean the folder above

student@server: ~/oslab-work

```
$ tree lab1
lab1
├── README.txt
├── task1_os_info.txt
├── task2_file_commands.txt
└── task2_files
```

you are inside task2_files

```
student@server:~/oslab-work/lab1/task2_files$ pwd > ../task2_file_commands.txt
```

From inside task2_files, .. is lab1. So ../task2_file_commands.txt is a file in lab1, not in task2_files.

Check where you are with pwd before every command that writes a file.

Make, write and record

student@server: ~/oslab-work/lab1

```
$ mkdir task2_files
$ cd task2_files
$ pwd > ../task2_file_commands.txt
$ ls >> ../task2_file_commands.txt
$ touch FILE1.txt FILE2.txt
$ ls >> ../task2_file_commands.txt
$ echo "This is file FILE1" > FILE1.txt
$ echo "This is file FILE2" > FILE2.txt
$ cat FILE1.txt FILE2.txt >> ../task2_file_commands.txt
```

Replace FILE1 and FILE2, shown in orange, with your own file1 and file2.

Every result goes into the record file. The first line uses >, the rest use >>.

Copy, rename and delete

student@server: ~/oslab-work/lab1/task2_files

```
$ cp FILE1.txt FILE1_copy.txt
$ ls >> ../task2_file_commands.txt
$ mv FILE2.txt FILE2_renamed.txt
$ ls >> ../task2_file_commands.txt
$ rm FILE1_copy.txt
$ ls >> ../task2_file_commands.txt
$ cd ..
$ cat task2_file_commands.txt
```

At the end you have

student@server: ~/oslab-work/lab1

```
$ tree task2_files
task2_files
├── FILE1.txt
└── FILE2_renamed.txt

0 directories, 2 files
```

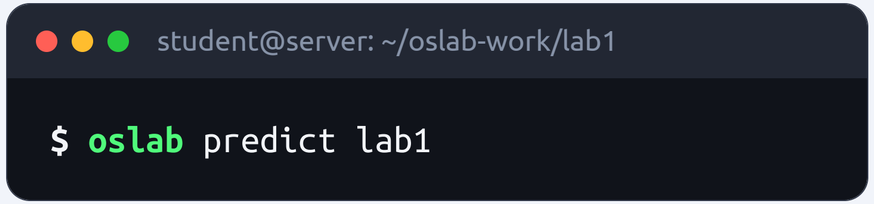
PREDICT

Guess first, then try

Write down what you expect before you run the next tasks.

Four questions, before you run anything

1. You start COUNT copies of sleep in the background. How many lines with sleep does ps show?
2. They have all finished. Is the program file sleep still on the disk?
3. Does apt-get remove delete the package's settings in /etc?
4. One sentence: what is the difference between a program and a process?



PREDICTION

The rules

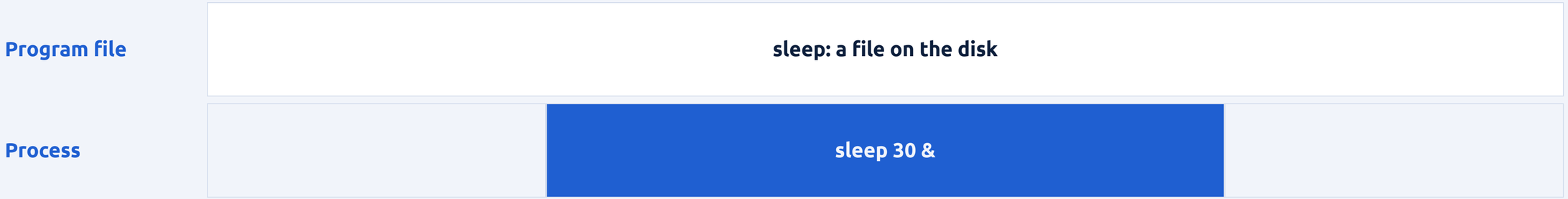
Alone, and without AI. The first answer is the one that counts. A wrong guess costs nothing.

TASK 4

Program or process?

A program is a file. A process is a program that is running.

A program stays. A process comes and goes.



which sleep finds the file before, during and after. ps shows a sleep line only while the process runs.

Run it in the background

student@server: ~/oslab-work/lab1

```
$ which sleep
$ sleep 30 &
$ ps > task4_process_list.txt
$ cat task4_process_list.txt
```

thirty seconds later

```
$ ps
$ which sleep
```

- The & at the end runs the command in the background and gives you the prompt back.
- ps lists the processes of your terminal.
- Afterwards ps shows no sleep, but which sleep still finds the file.

For your report: what did you see before, during and after?

Task 5: several at once

student@server: ~/oslab-work/lab1

```
$ sleep 300 &                # type this line COUNT times
$ ps > task5_multitasking.txt
$ cat task5_multitasking.txt
```

The up-arrow key brings the last line back.

FOR YOUR REPORT

Then take a screenshot

Keep it as task5.png. Your terminal prompt and the ps result must both be visible.

ps now shows COUNT lines with sleep: several processes at the same time.

What multitasking really is



The system switches between processes so fast that they seem to run together. A sleeping process uses no processor, so your instructor shows one that does.

TASK 6

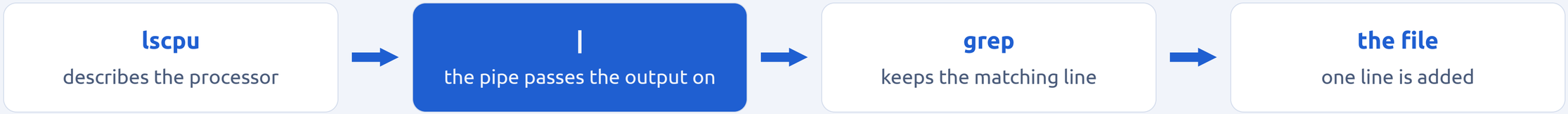
Is this a virtual machine?

Ask the system which hardware it thinks it runs on.

Four commands, one file

student@server: ~/oslab-work/lab1

```
$ systemd-detect-virt > task6_virtualization_check.txt
$ lscpu | grep -i "hypervisor vendor" >> task6_virtualization_check.txt
$ uname -r >> task6_virtualization_check.txt
$ hostname >> task6_virtualization_check.txt
$ cat task6_virtualization_check.txt
```



Reading the result

First line	What it means
kvm, vmware, or another name	A virtual machine. The hypervisor vendor line names the program that runs it.
wsl	Linux running inside Windows. It is also a virtual environment.
none	Real hardware: no hypervisor was found.

Take a screenshot as task6.png, then run oslab check lab1. All four milestones must show PASS.

If you finish early

OPTIONAL

Plus

Run top. The line that starts with Tasks counts every process on the server. Run nproc: it prints the number of processors. Compare both with your sleep lines.

OPTIONAL

Challenge

ps -e lists every process of the server, and wc -l counts lines. How many are running? You started only a few. Who started the others?

Neither is needed for full marks.

LIVE CHECKPOINT

Show what you can do alone

Fifteen minutes near the end of the lab. Worth 3 of the 10 points.

Why is there a checkpoint?

It shows what you can do alone

For most of the lab you may use hints and AI tools. The checkpoint is the one moment without them. It tells you, and the instructor, which commands you really know.

Your numbers are new

The names and numbers change at the checkpoint. An answer copied from a neighbour, or from your own earlier output, does not fit.

The instructor sees who needs help

Your answers arrive at once. The instructor can explain a common mistake before the next lab builds on this one.

It is fair

Everyone gets the same kind of task. You can take it even if an earlier task is not finished.

What happens, step by step

1. The instructor opens it
Close your AI tools

→

2. oslab checkpoint lab1
Answer five questions

→

3. A small file task
Folder, file, copy, rename, list

→

4. oslab check lab1
The fifth milestone shows PASS

Before it opens

The command only says: the checkpoint is not open yet. Wait for the instructor.

While you work

Use your terminal and what you learned in this lab. No AI tools, and no help from a neighbour.

When you finish

oslab check lab1 tells you whether the file task is right. You can fix it and check again.

What you see when you run it

example: your numbers will be different

```
$ oslab checkpoint lab1
Checkpoint: your own work. Close AI tools; do not ask a neighbour.
Work alone, with AI tools closed. Use only commands from this lab.

You start 3 copies of `sleep 300` in the background, then run `ps`.
How many lines with `sleep` does `ps` show?
> 3

Type the kernel release of this server: the number that `uname -r` prints.
> your answer
```

The five questions

1. How many sleep lines ps shows
2. Whether the program file is still there afterwards
3. The kernel release of the server
4. How many files the file task leaves
5. One sentence: program and process

Type each answer after the > and press Enter. Your answers are saved once: the first answer counts.

Then the file task

example: your names will be different

```
Saved and sent to the instructor.

Now do it for real, in your lab1 folder:
  1. Make a folder named cp-forest and go into it.
  2. Create a file named maple.txt that contains the text: hello forest
  3. Copy it to a file named maple-copy.txt.
  4. Rename maple.txt to maple-old.txt.
  5. Go back to the lab1 folder and save the list of files
     of cp-forest in checkpoint.txt
Then run: oslab check lab1
```

You already know how

These are the commands of Task 2: mkdir, cd, echo with >, cp, mv and ls with >.

The guide does not show the commands here. Writing them yourself is the test.

How the checkpoint is marked

Part	Points	Who marks it, and how
Your answers to the questions	1	The computer. All right is 1 point. At least half right is half a point.
The file task	1	The computer. oslab check lab1 looks at your folder and your files.
Your one sentence	1	The instructor reads it.

Can I answer again?

No. Your answers are saved once. You can still fix the file task and run oslab check lab1 again.

I did not finish Task 6

Take the checkpoint anyway. It does not depend on your earlier files.

PART B · HOMEWORK

Install, remove, purge

On your own Ubuntu. Due before the next class. Worth 3 points.

How your work travels



No Word or PDF reports: you use Markdown and Git. The server pushes first, your computer pushes second, and after that the server only pulls.

Step 1: on the server, push your Part A files

student@server: ~

```
$ mkdir -p ~/os-gic-YOUR_ID/os-lab-YOUR_ID/lab1
$ cp ~/oslab-work/lab1/task*.txt ~/os-gic-YOUR_ID/os-lab-YOUR_ID/lab1/
$ cp -r ~/oslab-work/lab1/task2_files ~/os-gic-YOUR_ID/os-lab-YOUR_ID/lab1/
$ cd ~/os-gic-YOUR_ID
$ git init
$ git add .
$ git commit -m "Lab 1: Part A files"
$ git branch -M main
$ git remote add origin https://github.com/YOUR_USERNAME/OS-GIC-YOUR_ID.git
$ git push -u origin main
```

- First create an empty repository named OS-GIC-YOUR_ID on GitHub.
- The first time, tell Git your name and email with git config.
- GitHub asks for a personal access token, not your password.

Do this at the end of the class if there is time, so the instructor can help.

Step 2: on your own Ubuntu, clone

you@your-ubuntu: ~

```
$ git clone https://github.com/YOUR_USERNAME/OS-GIC-YOUR_ID.git os-gic-YOUR_ID
$ cd os-gic-YOUR_ID/os-lab-YOUR_ID/lab1
$ mkdir images
$ ls
```

ls shows the files you made on the server. They travelled through GitHub.

Stay in this lab1 folder for Task 3, so its result file is saved in the right place.

Task 3: what each command does

Command	What it does
`sudo apt-get update`	Refreshes the list of available software. Installs nothing.
`sudo apt-get install mc`	Downloads and installs the package.
`sudo apt-get remove mc`	Uninstalls the program. Keeps its settings in /etc.
`sudo apt-get purge mc`	Uninstalls the program and deletes its settings.

mc is Midnight Commander. Its settings live in the folder /etc/mc.

What happens to /etc/mc

Step	start	install	remove	purge
Program	absent	installed	gone	gone
Settings	absent	present	still there	gone

Remove keeps the settings. Only purge deletes them. Compare this with your prediction.

Task 3: the three steps

you@your-ubuntu: ~/os-gic-YOUR_ID/os-lab-YOUR_ID/lab1

```
$ sudo apt-get update
$ sudo apt-get install mc -y
$ which mc > task3_apt.txt
$ ls -ld /etc/mc >> task3_apt.txt
$ sudo apt-get remove mc -y
$ ls -ld /etc/mc >> task3_apt.txt
$ sudo apt-get purge mc -y
$ ls -ld /etc/mc >> task3_apt.txt 2>&1
$ cat task3_apt.txt
```

2>&1 also saves the error message when the folder is gone.

Then take a screenshot of the last ls -ld /etc/mc and keep it as task3.png.

Three screenshots, in the images folder

TASK 3

task3.png

The ls -ld /etc/mc after the purge.

TASK 5

task5.png

The ps list with your sleep lines.

TASK 6

task6.png

The four commands of Task 6 and their result.

Every screenshot must show your terminal prompt, so it is clear whose work it is. Then fill in README.md from the report template.

Steps 3 and 4: push, then pull on the server

Step 3: on your own Ubuntu

you@your-ubuntu: ~/os-gic-YOUR_ID/os-lab-YOUR_ID/lab1

```
$ cd ../..
$ git add .
$ git commit -m "Lab 1: Task 3, screenshots and report"
$ git push
```

Step 4: on the server

student@server: ~

```
$ cd ~/os-gic-YOUR_ID
$ git pull
$ tree
```

From now on, only pull on the server. Never edit your repository files there.

What tree must show

student@server: ~/os-gic-YOUR_ID

```
$ tree
os-gic-YOUR_ID
└── os-lab-YOUR_ID
    └── lab1
        ├── images
        │   ├── task3.png
        │   ├── task5.png
        │   └── task6.png
        ├── README.md
        ├── task1_os_info.txt
        ├── task2_file_commands.txt
        ├── task2_files
        │   ├── FILE1.txt
        │   └── FILE2_renamed.txt
        ├── task3_apt.txt
        ├── task4_process_list.txt
        ├── task5_multitasking.txt
        └── task6_virtualization_check.txt
4 directories, 12 files
```

Check it on the server

After git pull, the tree must match this, with your own ID and your own file names.

Folders are shown in blue. The last line counts the folders and the files.

The course website checks these names to show your progress.

How Lab 1 is graded

Evidence	Points	Where
Files: oslab check lab1 passes the four milestones	2	In class
Report: kernel and distribution, remove and purge, program and process, virtual machine	2	Homework
Prediction saved in time, and honestly corrected	2	In class
Live checkpoint: answers, the file task and one sentence	3	In class
The APT file, three screenshots and README, pushed and pulled on time	1	Homework

Plus and Challenge are not needed for full marks.

Common mistakes

Mistake	What happens	Fix
Typing FILE1 literally	A file named FILE1.txt appears	Use your own file1 from oslab values lab1
> instead of >>	The record keeps only the last line	Use >> after the first line
Forgetting the &	The terminal waits and you cannot type	Press Ctrl+C, run it again with &
Running ps too late	There is no sleep line	Run ps right after you start sleep
Working in the wrong folder	No such file or directory	Check with pwd and ls

Key takeaways

01

Kernel and distribution

Linux is the kernel. Ubuntu is the kernel plus many other programs.

02

Program and process

A program is a file. A process is a running copy of it.

03

Remove and purge

Remove keeps the settings in /etc. Purge deletes them.

04

One arrow or two

> replaces a file. >> adds to its end.

Next lab

COMING UP

Lab 2: navigation and files

Absolute and relative paths, moving and copying files, and proving that a copy is identical.

SOURCE

Reference

The Lab 1 instruction and report template in the course repository, the Week 1 lecture notes, and man uname, man ls and man apt-get.